

Practical Session 5

Task 1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
j_rakshitha@RAKSHITHA:~$ cd ~/ossp_project
j_rakshitha@RAKSHITHA:~/ossp_project$ pwd
ls
/home/j_rakshitha/ossp_project
Makefile build copied.txt include sample.txt src
j_rakshitha@RAKSHITHA:~/ossp_project$ ls -l src
total 112
-rw-r--r-- 1 j_rakshitha j_rakshitha 1860 Aug 24 05:02 builtin_shell.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 969 Aug 24 04:02 child_exec.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1192 Aug 24 05:38 child_process_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1611 Aug 24 05:34 double_quote_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1257 Aug 24 03:56 dquote_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 764 Aug 24 04:00 escape_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1568 Aug 24 05:36 escape_sequence_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1365 Aug 18 05:20 file_copy.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 959 Aug 18 05:07 fork_exec.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 4275 Aug 18 03:43 history_shell.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1772 Aug 18 05:05 keyboard_shell.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 101 Aug 18 04:37 main.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2055 Aug 18 03:48 memory_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 652 Aug 18 04:38 myshell.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 3616 Aug 24 05:22 parser.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1881 Aug 24 04:21 path_resolver.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 794 Aug 17 05:39 process_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 939 Aug 18 05:55 process_states.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 904 Aug 24 05:31 single_quote_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 872 Aug 24 03:47 single_quotes.c+
-rw-r--r-- 1 j_rakshitha j_rakshitha 909 Aug 24 03:53 squote_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2495 Aug 24 05:28 syntax_parser.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1850 Aug 18 04:03 tokenizer.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2673 Aug 24 05:00 variable_expand.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1504 Aug 18 06:02 wait_waitpid.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1756 Aug 24 04:13 waitpid_monitor.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 900 Aug 18 06:04 zombie_demo.c
j_rakshitha@RAKSHITHA:~/ossp_project$ ^C
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/pipe_producer_consumer.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/pipe_producer_consum
er.c -o build/pipe_producer_consumer
```

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/pipe_producer_consumer.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/pipe_producer_consumer.c -o build/pipe_producer_consumer
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/pipe_producer_consumer
Producer-Consumer Communication using Anonymous Pipe
```

Consumer (Child) Process
Child PID: 562
Messages received: 705
Consumption completed successfully.

Producer (Parent) Process
Parent PID: 561
Messages generated: 1000
Generation completed successfully.

Communication Efficiency
Total communication time: 0.000929 seconds
Approximate throughput: 1076296.51 messages/second

Producer-Consumer communication completed successfully.

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/pipe_producer_consumer.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/pipe_producer_consumer.c -o build/pipe_producer_consumer
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/pipe_producer_consumer
Producer-Consumer Communication using Anonymous Pipe
```

Consumer (Child) Process
Child PID: 655
Messages received: 1000
Consumption completed successfully.

Producer (Parent) Process
Parent PID: 654
Messages generated: 1000
Generation completed successfully.

Communication Efficiency
Total communication time: 0.000830 seconds
Approximate throughput: 1205267.99 messages/second

Producer-Consumer communication completed successfully.

```
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <time.h>

#define COUNT 1000
#define MESSAGE_SIZE 100

int main()
{
    int pipefd[2];
    pid_t pid;
    char message[MESSAGE_SIZE];

    struct timespec start, end;

    if (pipe(pipefd) == -1)
    {
        perror("pipe failed");
        return 1;
    }

    printf("Producer-Consumer Communication using Anonymous Pipe\n\n");

    clock_gettime(CLOCK_MONOTONIC, &start);

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        /* Child process - Consumer */
        close(pipefd[1]);
```

```
while (received < COUNT)
{
    ssize_t bytes_read = 0;

    while (bytes_read < MESSAGE_SIZE)
    {
        ssize_t n = read(pipefd[0],
                        message + bytes_read,
                        MESSAGE_SIZE - bytes_read);

        if (n <= 0)
        {
            perror("read failed");
            close(pipefd[0]);
            exit(1);
        }

        bytes_read += n;
    }

    received++;
}

close(pipefd[0]);

printf("Consumer (Child) Process\n");
printf("Child PID: %d\n", getpid());
printf("Messages received: %d\n", received);
printf("Consumption completed successfully.\n");

exit(0);
}

else
{
    /* Parent process - Producer */
    close(pipefd[0]);

    for (int i = 1; i <= COUNT; i++)
    {
        memset(message, 0, MESSAGE_SIZE);
```

```
{
    /* Parent process - Producer */
    close(pipefd[0]);

    for (int i = 1; i <= COUNT; i++)
    {
        memset(message, 0, MESSAGE_SIZE);

        snprintf(message,
                 MESSAGE_SIZE,
                 "Data packet %d",
                 i);

        ssize_t bytes_written = 0;

        while (bytes_written < MESSAGE_SIZE)
        {
            ssize_t n = write(pipefd[1],
                              message + bytes_written,
                              MESSAGE_SIZE - bytes_written);

            if (n <= 0)
            {
                perror("write failed");
                close(pipefd[1]);
                return 1;
            }

            bytes_written += n;
        }

        close(pipefd[1]);

        waitpid(pid, NULL, 0);

        clock_gettime(CLOCK_MONOTONIC, &end);

        double elapsed =
            (end.tv_sec - start.tv_sec) +
            (end.tv_nsec - start.tv_nsec) / 1e9;
    }
}
```

```
GNU nano 8.7.1 src/pipe_producer_consumer.c *
    {
        perror("write failed");
        close(pipefd[1]);
        return 1;
    }

    bytes_written += n;
}

close(pipefd[1]);

waitpid(pid, NULL, 0);

clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1e9;

printf("\nProducer (Parent) Process\n");
printf("Parent PID: %d\n", getpid());
printf("Messages generated: %d\n", COUNT);
printf("Generation completed successfully.\n");

printf("\nCommunication Efficiency\n");
printf("Total communication time: %.6f seconds\n",
    elapsed);

if (elapsed > 0)
{
    printf("Approximate throughput: %.2f messages/second\n",
        COUNT / elapsed);
}

printf("\nProducer-Consumer communication completed successfully.\n");
}

return 0;
}
```

Task 2

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```

j_rakshitha@RAKSHITHA:~$ cd ~/ossp_project
nano src/pipeline_demo.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/pipeline_demo.c -o build/pipeline_demo
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/pipeline_demo
Pipeline Demonstration
Command: ls -l src | grep "\.c"

-rw-r--r-- 1 j_rakshitha j_rakshitha 1860 Aug 24 05:02 builtin_shell.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 969 Aug 24 04:02 child_exec.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1192 Aug 24 05:38 child_process_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1611 Aug 24 05:34 double_quote_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1257 Aug 24 03:56 dquote_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 764 Aug 24 04:00 escape_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1568 Aug 24 05:36 escape_sequence_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1365 Aug 18 05:20 file_copy.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1671 Aug 24 06:03 file_filter_pipe.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 959 Aug 18 05:07 fork_exec.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 4275 Aug 18 03:43 history_shell.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1772 Aug 18 05:05 keyboard_shell.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 101 Aug 18 04:37 main.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2055 Aug 18 03:48 memory_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 652 Aug 18 04:38 myshell.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 3616 Aug 24 05:22 parser.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1881 Aug 24 04:21 path_resolver.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1901 Aug 24 06:01 pipe_ls_grep.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 3147 Aug 24 05:56 pipe_producer_consumer.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1555 Aug 24 06:25 pipeline_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 794 Aug 17 05:39 process_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 939 Aug 18 05:55 process_states.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 904 Aug 24 05:31 single_quote_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 872 Aug 24 03:47 single_quotes.c+
-rw-r--r-- 1 j_rakshitha j_rakshitha 909 Aug 24 03:53 squote_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2495 Aug 24 05:28 syntax_parser.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1850 Aug 18 04:03 tokenizer.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2673 Aug 24 05:00 variable_expand.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1504 Aug 18 06:02 wait_waitpid.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1756 Aug 24 04:13 waitpid_monitor.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 900 Aug 18 06:04 zombie_demo.c

Pipeline completed successfully.
pipe() : Used
fork() : Used twice
dup2() : Used
exec() : Used
waitpid(): Used
j_rakshitha@RAKSHITHA:~/ossp_project$

```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t child1, child2;

    printf("Pipeline Demonstration\n");
    printf("Command: ls -l src | grep '\\.c'\n\n");

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    /* Child 1: ls -l src */
    child1 = fork();

    if (child1 == -1)
    {
        perror("fork");
        return 1;
    }

    if (child1 == 0)
    {
        close(pipefd[0]);

        if (dup2(pipefd[1], STDOUT_FILENO) == -1)
        {
            perror("dup2");
            exit(1);
        }

        close(pipefd[1]);

        execlp("ls", "ls", "-l", "src", (char *)NULL);

        perror("exec ls");
        exit(1);
    }
}
```

```

/* Child 2: grep "\.c" */
child2 = fork();

if (child2 == -1)
{
    perror("fork");
    return 1;
}

if (child2 == 0)
{
    close(pipefd[1]);

    if (dup2(pipefd[0], STDIN_FILENO) == -1)
    {
        perror("dup2");
        exit(1);
    }

    close(pipefd[0]);

    execlp("grep", "grep", "\\\.c", (char *)NULL);

    perror("exec grep");
    exit(1);
}

/* Parent */
close(pipefd[0]);
close(pipefd[1]);

waitpid(child1, NULL, 0);
waitpid(child2, NULL, 0);

printf("\nPipeline completed successfully.\n");
printf("pipe() : Used\n");
printf("fork() : Used twice\n");
printf("dup2() : Used\n");
printf("exec() : Used\n");
printf("waitpid(): Used\n");

return 0;
}

```